
13.4 Deletion

Like the other basic operations on an n -node red-black tree, deletion of a node takes $O(\lg n)$ time. Deleting a node from a red-black tree is more complicated than inserting a node.

The procedure for deleting a node from a red-black tree is based on the TREE-DELETE procedure on page 325. First, we need to customize the TRANSPLANT subroutine on page 324 that TREE-DELETE calls so that it applies to a red-black tree. Like TRANSPLANT, the new procedure RB-TRANSPLANT replaces the subtree rooted at node u by the subtree rooted at node v . The RB-TRANSPLANT procedure differs from TRANSPLANT in two ways. First, line 1 references the sentinel $T.nil$ instead of NIL . Second, the assignment to $v.p$ in line 6 occurs unconditionally: the procedure can assign to $v.p$ even if v points to the sentinel. We'll take advantage of the ability to assign to $v.p$ when $v = T.nil$.

RB-TRANSPLANT(T, u, v)

```
1 if  $u.p == T.nil$ 
2    $T.root = v$ 
3 elseif  $u == u.p.left$ 
4    $u.p.left = v$ 
5 else  $u.p.right = v$ 
6  $v.p = u.p$ 
```

The procedure RB-DELETE on the next page is like the TREE-DELETE procedure, but with additional lines of pseudocode. The additional lines deal with nodes x and y that may be involved in violations of the red-black properties. When the node z being deleted has at most one child, then y will be z . When z has two children, then, as in TREE-DELETE, y will be z 's successor, which has no left child and moves into z 's position in the tree. Additionally, y takes on z 's color. In either case, node y has at most one child: node x , which takes y 's place in the tree. (Node x will be the sentinel $T.nil$ if y has no children.) Since node y will be either removed from the tree or moved within the tree, the

procedure needs to keep track of y 's original color. If the red-black properties might be violated after deleting node z , RB-DELETE calls the auxiliary procedure RB-DELETE-FIXUP, which changes colors and performs rotations to restore the red-black properties.

Although RB-DELETE contains almost twice as many lines of pseudocode as TREE-DELETE, the two procedures have the same basic structure. You can find each line of TREE-DELETE within RB-DELETE (with the changes of replacing NIL by $T.nil$ and replacing calls to TRANSPLANT by calls to RB-TRANSPLANT), executed under the same conditions.

In detail, here are the other differences between the two procedures:

- Lines 1 and 9 set node y as described above: line 1 when node z has at most one child and line 9 when z has two children.
- Because node y 's color might change, the variable y -original-color stores y 's color before any changes occur. Lines 2 and 10 set this variable immediately after assignments to y . When node z has two children, then nodes y and z are distinct. In this case, line 17 moves y into z 's original position in the tree (that is, z 's location in the tree at the time RB-DELETE was called), and line 20 gives y the same color as z . When node y was originally black, removing or moving it could cause violations of the red-black properties, which are corrected by the call of RB-DELETE-FIXUP in line 22.

RB-DELETE(T, z)

```
1  $y = z$ 
2  $y$ -original-color =  $y$ .color
3 if  $z$ .left ==  $T.nil$ 
4    $x = z$ .right
5   RB-TRANSPLANT( $T, z, z$ .right) // replace  $z$  by its right child
6 elseif  $z$ .right ==  $T.nil$ 
7    $x = z$ .left
8   RB-TRANSPLANT( $T, z, z$ .left) // replace  $z$  by its left child
9 else  $y = \text{TREE-MINIMUM}(z$ .right) //  $y$  is  $z$ 's successor
10  $y$ -original-color =  $y$ .color
```

```

11  x = y.right
12  if y ≠ z.right                                // is y farther down the tree?
13      RB-TRANSPLANT(T, y, // replace y by its right child
                     y.right)
14      y.right = z.right                            // z's right child becomes
15      y.right.p = y                                // y's right child
16  else x.p = y                                    // in case x is T.nil
17      RB-TRANSPLANT(T, z, y)                    // replace z by its successor y
18      y.left = z.left                              // and give z's left child to y,
19      y.left.p = y                                  // which had no left child
20      y.color = z.color
21 if y-original-color == BLACK // if any red-black violations
                                occurred,
22  RB-DELETE-FIXUP(T, x) // correct them

```

- As discussed, the procedure keeps track of the node x that moves into node y 's original position at the time of call. The assignments in lines 4, 7, and 11 set x to point to either y 's only child or, if y has no children, the sentinel $T.nil$.
- Since node x moves into node y 's original position, the attribute $x.p$ must be set correctly. If node z has two children and y is z 's right child, then y just moves into z 's position, with x remaining a child of y . Line 12 checks for this case. Although you might think that setting $x.p$ to y in line 16 is unnecessary since x is a child of y , the call of RB-DELETE-FIXUP relies on $x.p$ being y even if x is $T.nil$. Thus, when z has two children and y is z 's right child, executing line 16 is necessary if y 's right child is $T.nil$, and otherwise it does not change anything.

Otherwise, node z is either the same as node y or it is a proper ancestor of y 's original parent. In these cases, the calls of RB-TRANSPLANT in lines 5, 8, and 13 set $x.p$ correctly in line 6 of RB-TRANSPLANT. (In these calls of RB-TRANSPLANT, the third parameter passed is the same as x .)

- Finally, if node y was black, one or more violations of the red-black properties might arise. The call of **RB-DELETE-FIXUP** in line 22 restores the red-black properties. If y was red, the red-black properties still hold when y is removed or moved, for the following reasons:
 1. No black-heights in the tree have changed. (See Exercise 13.4-1.)
 2. No red nodes have been made adjacent. If z has at most one child, then y and z are the same node. That node is removed, with a child taking its place. If the removed node was red, then neither its parent nor its children can also be red, so moving a child to take its place cannot cause two red nodes to become adjacent. If, on the other hand, z has two children, then y takes z 's place in the tree, along with z 's color, so there cannot be two adjacent red nodes at y 's new position in the tree. In addition, if y was not z 's right child, then y 's original right child x replaces y in the tree. Since y is red, x must be black, and so replacing y by x cannot cause two red nodes to become adjacent.
 3. Because y could not have been the root if it was red, the root remains black.

If node y was black, three problems may arise, which the call of **RB-DELETE-FIXUP** will remedy. First, if y was the root and a red child of y became the new root, property 2 is violated. Second, if both x and its new parent are red, then a violation of property 4 occurs. Third, moving y within the tree causes any simple path that previously contained y to have one less black node. Thus, property 5 is now violated by any ancestor of y in the tree. We can correct the violation of property 5 by saying that when the black node y is removed or moved, its blackness transfers to the node x that moves into y 's original position, giving x an “extra” black. That is, if we add 1 to the count of black nodes on any simple path that contains x , then under this interpretation, property 5 holds. But now another problem emerges: node x is neither red nor black, thereby violating property 1. Instead, node x is either “doubly

black” or “red-and-black,” and it contributes either 2 or 1, respectively, to the count of black nodes on simple paths containing x . The *color* attribute of x will still be either RED (if x is red-and-black) or BLACK (if x is doubly black). In other words, the extra black on a node is reflected in x ’s pointing to the node rather than in the *color* attribute.

The procedure RB-DELETE-FIXUP on the next page restores properties 1, 2, and 4. Exercises 13.4-2 and 13.4-3 ask you to show that the procedure restores properties 2 and 4, and so in the remainder of this section, we focus on property 1. The goal of the **while** loop in lines 1–43 is to move the extra black up the tree until

1. x points to a red-and-black node, in which case line 44 colors x (singly) black;
2. x points to the root, in which case the extra black simply vanishes; or
3. having performed suitable rotations and recolorings, the loop exits.

Like RB-INSERT-FIXUP, the RB-DELETE-FIXUP procedure handles two symmetric situations: lines 3–22 for when node x is a left child, and lines 24–43 for when x is a right child. Our proof focuses on the four cases shown in lines 3–22.

Within the **while** loop, x always points to a nonroot doubly black node. Line 2 determines whether x is a left child or a right child of its parent $x.p$ so that either lines 3–22 or 24–43 will execute in a given iteration. The sibling of x is always denoted by a pointer w . Since node x is doubly black, node w cannot be *T.nil*, because otherwise, the number of blacks on the simple path from $x.p$ to the (singly black) leaf w would be smaller than the number on the simple path from $x.p$ to x .

Recall that the RB-DELETE procedure always assigns to $x.p$ before calling RB-DELETE-FIXUP (either within the call of RB-TRANSPLANT in line 13 or the assignment in line 16), even when node x is the sentinel *T.nil*. That is because RB-DELETE-FIXUP references x ’s parent $x.p$ in several places, and this attribute must point to the node that became x ’s parent in RB-DELETE—even if x is *T.nil*.

Figure 13.7 demonstrates the four cases in the code when node x is a left child. (As in RB-INSERT-FIXUP, the cases in RB-DELETE-FIXUP are not mutually exclusive.) Before examining each case in detail, let's look more generally at how we can verify that the transformation in each of the cases preserves property 5. The key idea is that in each case, the transformation applied preserves the number of black nodes (including x 's extra black) from (and including) the root of the subtree shown to the roots of each of the subtrees $\alpha, \beta, \dots, \zeta$. Thus, if property 5 holds prior to the transformation, it continues to hold afterward. For example, in Figure 13.7(a), which illustrates case 1, the number of black nodes from the root to the root of either subtree α or β is 3, both before and after the transformation. (Again, remember that node x adds an extra black.) Similarly, the number of black nodes from the root to the root of any of γ, δ, ϵ , and ζ is 2, both before and after the transformation.² In Figure 13.7(b), the counting must involve the value c of the *color* attribute of the root of the subtree shown, which can be either RED or BLACK.

RB-DELETE-FIXUP(T, x)

```

1 while  $x \neq T.root$  and  $x.color == BLACK$ 
2   if  $x == x.p.left$                                      // is  $x$  a left
                                                            child?
3      $w = x.p.right$                                        //  $w$  is  $x$ 's
                                                            sibling
4     if  $w.color == RED$ 
5        $w.color = BLACK$ 
6        $x.p.color = RED$ 
7       LEFT-ROTATE( $T, x.p$ )
8        $w = x.p.right$ 
9     if  $w.left.color == BLACK$  and  $w.right.color ==$ 
       BLACK
10       $w.color = RED$ 
11       $x = x.p$ 
12   else
13     if  $w.right.color == BLACK$ 

```

} case 1

} case 2

```

14      w.left.color = BLACK
15      w.color = RED
16      RIGHT-ROTATE(T, w)
17      w = x.p.right
18      w.color = x.p.color
19      x.p.color = BLACK
20      w.right.color = BLACK
21      LEFT-ROTATE(T, x.p)
22      x = T.root
23  else // same as lines 3–22, but with “right” and “left” exchanged
24      w = x.p.left
25      if w.color == RED
26          w.color = BLACK
27          x.p.color = RED
28          RIGHT-ROTATE(T, x.p)
29          w = x.p.left
30      if w.right.color == BLACK and w.left.color == BLACK
31          w.color = RED
32          x = x.p
33      else
34          if w.left.color == BLACK
35              w.right.color = BLACK
36              w.color = RED
37              LEFT-ROTATE(T, w)
38              w = x.p.left
39              w.color = x.p.color
40              x.p.color = BLACK
41              w.left.color = BLACK
42              RIGHT-ROTATE(T, x.p)
43              x = T.root
44  x.color = BLACK

```

case 3

case 4

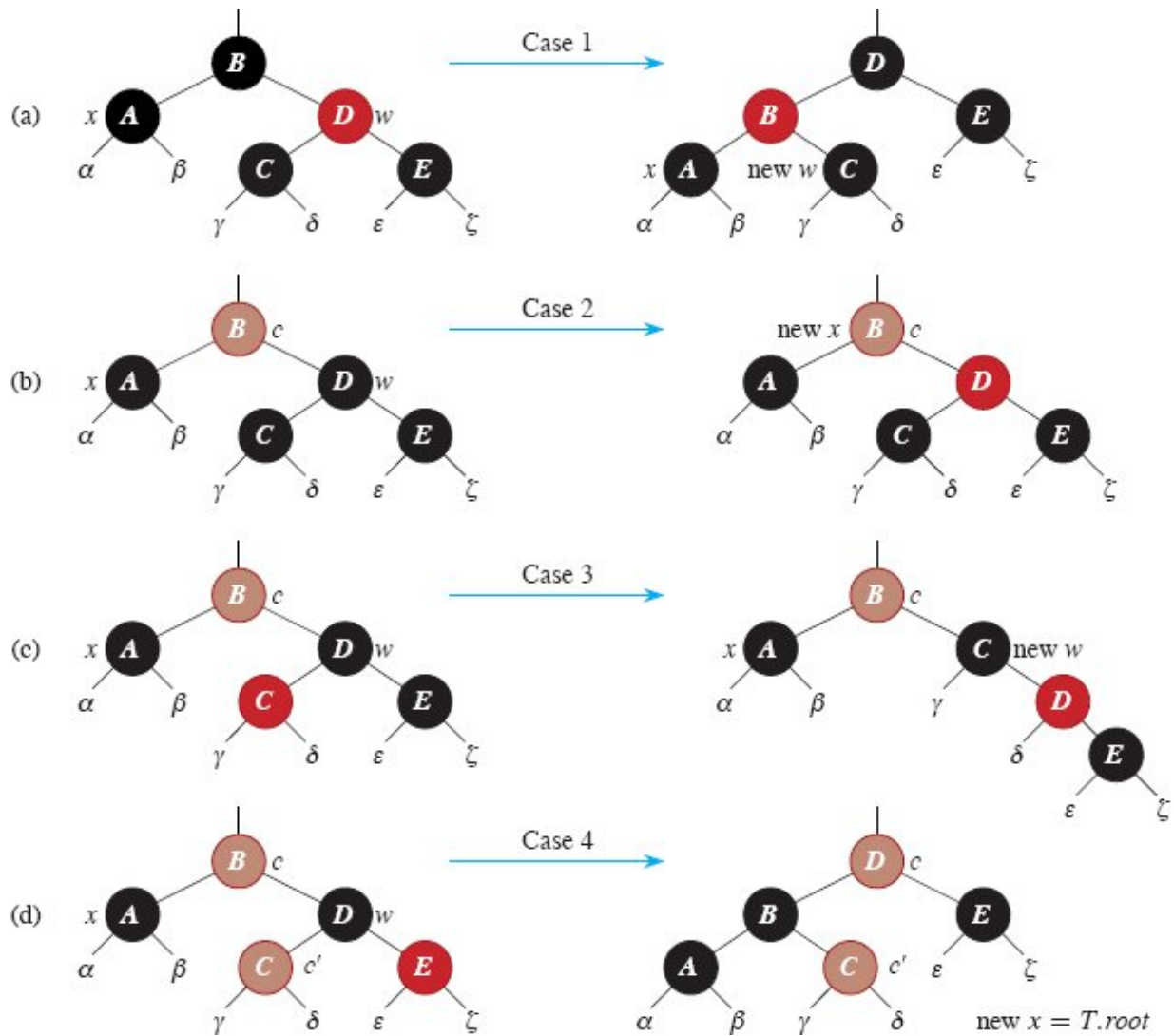


Figure 13.7 The cases in lines 3–22 of the procedure RB-DELETE-FIXUP. Brown nodes have *color* attributes represented by c and c' , which may be either RED or BLACK. The letters $\alpha, \beta, \dots, \zeta$ represent arbitrary subtrees. Each case transforms the configuration on the left into the configuration on the right by changing some colors and/or performing a rotation. Any node pointed to by x has an extra black and is either doubly black or red-and-black. Only case 2 causes the loop to repeat. **(a)** Case 1 is transformed into case 2, 3, or 4 by exchanging the colors of nodes B and D and performing a left rotation. **(b)** In case 2, the extra black represented by the pointer x moves up the tree by coloring node D red and setting x to point to node B . If case 2 is entered through case 1, the **while** loop terminates because the new node x is red-and-black, and therefore the value c of its *color* attribute is RED. **(c)** Case 3 is transformed to case 4 by exchanging the colors of nodes C and D and performing a right rotation. **(d)** Case 4 removes the extra black represented by x by changing some colors and performing a left rotation (without violating the red-black properties), and then the loop terminates.

If we define $\text{count}(\text{RED}) = 0$ and $\text{count}(\text{BLACK}) = 1$, then the number of black nodes from the root to α is $2 + \text{count}(c)$, both before and after the transformation. In this case, after the transformation, the new node x has *color* attribute c , but this node is really either red-and-black (if $c = \text{RED}$) or doubly black (if $c = \text{BLACK}$). You can verify the other cases similarly (see Exercise 13.4-6).

Case 1. x 's sibling w is red

Case 1 (lines 5–8 and Figure 13.7(a)) occurs when node w , the sibling of node x , is red. Because w is red, it must have black children. This case switches the colors of w and $x.p$ and then performs a left-rotation on $x.p$ without violating any of the red-black properties. The new sibling of x , which is one of w 's children prior to the rotation, is now black, and thus case 1 converts into one of cases 2, 3, or 4.

Cases 2, 3, and 4 occur when node w is black and are distinguished by the colors of w 's children.

Case 2. x 's sibling w is black, and both of w 's children are black

In case 2 (lines 10–11 and Figure 13.7(b)), both of w 's children are black. Since w is also black, this case removes one black from both x and w , leaving x with only one black and leaving w red. To compensate for x and w each losing one black, x 's parent $x.p$ can take on an extra black. Line 11 does so by moving x up one level, so that the **while** loop repeats with $x.p$ as the new node x . If case 2 enters through case 1, the new node x is red-and-black, since the original $x.p$ was red. Hence, the value c of the *color* attribute of the new node x is RED, and the loop terminates when it tests the loop condition. Line 44 then colors the new node x (singly) black.

Case 3. x 's sibling w is black, w 's left child is red, and w 's right child is black

Case 3 (lines 14–17 and Figure 13.7(c)) occurs when w is black, its left child is red, and its right child is black. This case switches the colors of w and its left child $w.\text{left}$ and then performs a right rotation on w without violating any of the red-black properties. The new sibling w of x is now a

black node with a red right child, and thus case 3 falls through into case 4.

Case 4. x 's sibling w is black, and w 's right child is red

Case 4 (lines 18–22 and Figure 13.7(d)) occurs when node x 's sibling w is black and w 's right child is red. Some color changes and a left rotation on $x.p$ allow the extra black on x to vanish, making it singly black, without violating any of the red-black properties. Line 22 sets x to be the root, and the **while** loop terminates when it next tests the loop condition.

Analysis

What is the running time of RB-DELETE? Since the height of a red-black tree of n nodes is $O(\lg n)$, the total cost of the procedure without the call to RB-DELETE-FIXUP takes $O(\lg n)$ time. Within RB-DELETE-FIXUP, each of cases 1, 3, and 4 lead to termination after performing a constant number of color changes and at most three rotations. Case 2 is the only case in which the **while** loop can be repeated, and then the pointer x moves up the tree at most $O(\lg n)$ times, performing no rotations. Thus, the procedure RB-DELETE-FIXUP takes $O(\lg n)$ time and performs at most three rotations, and the overall time for RB-DELETE is therefore also $O(\lg n)$.

Exercises

13.4-1

Show that if node y in RB-DELETE is red, then no black-heights change.

13.4-2

Argue that after RB-DELETE-FIXUP executes, the root of the tree must be black.

13.4-3

Argue that if in RB-DELETE both x and $x.p$ are red, then property 4 is restored by the call to RB-DELETE-FIXUP(T, x).

13.4-4

In Exercise 13.3-2 on page 346, you found the red-black tree that results from successively inserting the keys 41, 38, 31, 12, 19, 8 into an initially empty tree. Now show the red-black trees that result from the successive deletion of the keys in the order 8, 12, 19, 31, 38, 41.

13.4-5

Which lines of the code for RB-DELETE-FIXUP might examine or modify the sentinel $T.nil$?

13.4-6

In each of the cases of Figure 13.7, give the count of black nodes from the root of the subtree shown to the roots of each of the subtrees α , β , ..., ζ , and verify that each count remains the same after the transformation. When a node has a *color* attribute c or c' , use the notation $\text{count}(c)$ or $\text{count}(c')$ symbolically in your count.

13.4-7

Professors Skelton and Baron worry that at the start of case 1 of RB-DELETE-FIXUP, the node $x.p$ might not be black. If $x.p$ is not black, then lines 5–6 are wrong. Show that $x.p$ must be black at the start of case 1, so that the professors need not be concerned.

13.4-8

A node x is inserted into a red-black tree with RB-INSERT and then is immediately deleted with RB-DELETE. Is the resulting red-black tree always the same as the initial red-black tree? Justify your answer.

★ 13.4-9

Consider the operation $\text{RB-ENUMERATE}(T, r, a, b)$, which outputs all the keys k such that $a \leq k \leq b$ in a subtree rooted at node r in an n -node red-black tree T . Describe how to implement RB-ENUMERATE in $\Theta(m + \lg n)$ time, where m is the number of keys that are output. Assume that the keys in T are unique and that the values a and b appear as keys in T . How does your solution change if a and b might not appear in T ?

Problems

13-1 *Persistent dynamic sets*

During the course of an algorithm, you sometimes find that you need to maintain past versions of a dynamic set as it is updated. We call such a set *persistent*. One way to implement a persistent set is to copy the entire set whenever it is modified, but this approach can slow down a program and also consume a lot of space. Sometimes, you can do much better.

Consider a persistent set S with the operations INSERT, DELETE, and SEARCH, which you implement using binary search trees as shown in Figure 13.8(a). Maintain a separate root for every version of the set. In order to insert the key 5 into the set, create a new node with key 5. This node becomes the left child of a new node with key 7, since you cannot modify the existing node with key 7. Similarly, the new node with key 7 becomes the left child of a new node with key 8 whose right child is the existing node with key 10. The new node with key 8 becomes, in turn, the right child of a new root r' with key 4 whose left child is the existing node with key 3. Thus, you copy only part of the tree and share some of the nodes with the original tree, as shown in Figure 13.8(b).

Assume that each tree node has the attributes *key*, *left*, and *right* but no parent. (See also Exercise 13.3-6 on page 346.)

- a.** For a persistent binary search tree (not a red-black tree, just a binary search tree), identify the nodes that need to change to insert or delete a node.